

Getting data from server

- For a while now we have only been working on **"frontend", i.e. client-side (browser) functionality**. We will begin working on **"backend", i.e. server-side functionality** in the *third part of this course*. Nonetheless, we will now take a step in that direction by familiarizing ourselves with how the code executing in the browser communicates with the backend.
- we are going to use a tool meant to be used during software development called **JSON Server** to act as our **fake server**.
- so first create **db.json** file into root folder which I am working in **part2_c => getting-data-from-server** so create here all folders and files are present like **vite-setup,src**, so create here **db.json** here.

```
{
  "notes": [
    {
      "id": "1",
      "content": "HTML is easy",
      "important": true
    },
    {
      "id": "2",
      "content": "Browser can execute only JavaScript",
      "important": false
    },
    {
      "id": "3",
      "content": "GET and POST are the most important methods of HTTP protocol",
      "important": true
    }
  ]
}
```

- put this data into **db.json**.
- And start the fake server with help of tool called **JSON-Server**.
- **npx json-server --port 3001 db.json**.
- redirect into browser **localhost:3001/notes**.
- Extra info :-
 - (A "port" is just a numbered door on your computer. Your computer has exactly 65,535 doors.)
 - You should never use the port that your Vite frontend is running on (usually 5173 or 3000). If your frontend and backend try to share the exact same port, your computer will crash.
- Going forward, the idea will be to save the notes to the server, which in this case means saving them to the json-server. The **React code fetches** the notes from the **server** and **renders** them to the

screen. Whenever a new note is added to the application, the React code also sends it to the server to make the new note persist in "memory".

The browser as a runtime environment

The Problem: JavaScript only has one brain.(single-thread)

- JavaScript is **"single-threaded"**. Imagine JavaScript is a chef in a kitchen, but the chef only has one hand (single-thread). They can only do exactly one task at a time. If the chef puts a pizza in the oven and just stands there staring at it for 20 minutes waiting for it to finish, the entire restaurant shuts down. No other food gets cooked.
- If your React app asks the json-server for data, and it takes 3 seconds to download... if JavaScript just stood there waiting, your entire website would freeze for 3 seconds. You couldn't scroll, you couldn't click buttons, it would look like it crashed.

The Solution: Asynchronous Callbacks

- Because we don't want the website to freeze, JavaScript uses Asynchronous Code. Instead of staring at the oven, the chef puts the pizza in the oven, sets a timer, and goes back to chopping vegetables. When the timer rings (a "callback"), the chef knows the pizza is done and takes it out.
 - What this means for your code:
 - When you tell React to connect to your http://localhost:3001/notes server, you don't write it like normal code. You use a "Promise" (a timer).
 - You basically tell React: "Hey, go start downloading this data. But don't wait for it! Keep drawing the website. .then() when the data finally arrives, run this function to update the screen."
-

npm

1. What is NPM?

- NPM stands for Node Package Manager. Think of NPM as the App Store for developers.

You don't have to build every single tool from scratch. Thousands of smart developers have already written tools for you, and put them on NPM for free. When you run `npm install axios` in your terminal, it reaches out to the "App Store", downloads the code for Axios, and puts it in your massive `node_modules` folder. It also writes the name of it inside your `package.json` file, so your app remembers that you installed it!

- `npx` stands for **Node Package Execute**.
 - `npm` is for installing things permanently on your hard drive.
 - `npx` is for running things.
- **The Magic of `npx`**

- Normally, to run a program, you have to download it, install it, and keep it on your computer forever.
- **npx has a superpower:** if you ask it to run a tool that you do not own, it will secretly reach up to the internet, download a temporary copy of the tool, run it one time, and then delete it so it doesn't clutter your hard drive!

2. Why Axios instead of Fetch?

- `fetch()` is a tool that is built directly into every web browser. It works, but it is annoying and requires extra chores.
- When a database sends data across the internet, it sends it as a massive, ugly string of text (raw JSON). If you use `fetch()`, you have to write an extra line of code to manually translate that raw text back into a JavaScript array:

```
// The Fetch Way (Annoying)
fetch('http://localhost:3001/notes')
  .then(response => response.json()) // 🚨 Extra chore required here!
  .then(data => console.log(data))
```

- Axios is a tool you download from the NPM App Store that does the chores for you. Axios automatically translates the raw text into a beautiful JavaScript array behind the scenes. **It also handles errors (like 404 Not Found) much better than fetch does.**

```
// The Axios Way (Clean)
axios.get('http://localhost:3001/notes')
  .then(response => console.log(response.data)) // Automatically translated!
```

- Another good reason to use Axios is that it helps us get familiar with adding external libraries, or npm packages, to React projects.
- **npm install axios**

you will find this into dependencies - the dependencies part is of most interest to us as it defines what dependencies, or external libraries, the project has.

```
"dependencies": {
  **"axios": "^1.19.0",**
  "react": "^19.2.8",
  "react-dom": "^19.2.8"
},
```

- **npm install json-server --save-dev**

Install json-server as a development dependency (only used during development)

- and making a small addition to the scripts part of the package.json file:

```
{
  // ...
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "lint": "eslint .",
    "preview": "vite preview",
    **"server": "json-server -p 3001 db.json" >>> add this line into scripts**
  },
}
```

- We can now conveniently, without parameter definitions, start the json-server from the project root directory with the command:

npm run server this will generate Endpoints: <http://localhost:3001/notes>

Axios and promises

- how to get data using axios

```
import axios from 'axios'
const promise = axios.get('http://localhost:3001/notes')
```

- Axios' method get returns a promise
 - In other words, a promise is an object that represents an asynchronous operation. A promise can have three distinct states:
 - **The promise is pending:** It means that the asynchronous operation corresponding to the promise has not yet finished and the final value is not available yet.
 - **The promise is fulfilled:** It means that the operation has been completed and the final value is available, which generally is a successful operation.
 - **The promise is rejected:** It means that an error prevented the final value from being determined, which generally represents a failed operation.
- If we want to access the result of the operation represented by the promise, we must register an event handler to the promise. This is achieved using the method then:

```
const promise = axios.get('http://localhost:3001/notes')

promise.then(response => {
  console.log(response)
})
```

- Improvised version but still problematic

```
axios
  .get('http://localhost:3001/notes')
  .then(response => {
    const { data } = response;
    createRoot(document.getElementById('root')).render(
      <StrictMode>
        <App notes={data} />
      </StrictMode>,
    )
  })
```

- The course says: "What's not immediately obvious is where axios.get should be placed."
 - If you just copy-paste axios.get() directly into the middle of your App.jsx file, it will cause a catastrophic Infinite Loop. Every time you type a letter in an input box, React re-renders the page... which would trigger axios.get() again... which updates the state... which triggers a re-render... which triggers axios.get() again. Your computer will crash.

Effect-hooks (useEffect)

- As per the official docs:
 - Effects let a component connect to and synchronize with external systems. This includes dealing with network, browser DOM, animations, widgets written using a different UI library, and other non-React code.
 - As such, effect hooks are precisely the right tool to use when fetching data from a server.
- so we do need to pass data as prop to App in main.jsx remove all stuff except rednering block.
- Initial code block of hook (Syntax)

```
import { useState, useEffect } from "react";
import axios from "axios";

const App = () => {

  const [notes, setNotes] = useState([]);

  useEffect(() => {
    console.log('useEffect');
    axios
      .get('http://localhost:3001/notes')
      .then(response => {
        console.log('promise fulfilled');
        setNotes(response.data);
      })
  })
```

```
    }, []);

    console.log('render', notes.length, 'notes')
  };

  export default App;
```

- Output

```
render 0 notes
useEffect
promise fulfilled
render 3 notes
```

- First, the body of the function defining the component is executed and the component is rendered for the first time. At this point render 0 notes is printed, meaning data hasn't been fetched from the server yet.
 - The following function, or effect in React parlance:

```
() => {
  console.log('effect')
  axios
    .get('http://localhost:3001/notes')
    .then(response => {
      console.log('promise fulfilled')
      setNotes(response.data)
    })
}
```

- is executed immediately after rendering. The execution of the function results in effect being printed to the console, and the command *axios.get* initiates the fetching of data from the server **as well as registers the following function as an event handler for the operation:**
- When data arrives from the server, the JavaScript runtime calls the function registered as the event handler, which prints promise fulfilled to the console and stores the notes received from the server into the state using the function `setNotes(response.data)`;
- As always, a call to a state-updating function triggers the re-rendering of the component. As a result, render 3 notes is printed to the console, and the notes fetched from the server are rendered to the screen.
- Finally, let's take a look at the definition of the effect hook as a whole:

```
useEffect(() => {
  console.log('effect')
```

```

    axios
      .get('http://localhost:3001/notes').then(response => {
        console.log('promise fulfilled')
        setNotes(response.data)
      })
  }, [])

```

- Let's rewrite the code a bit differently.

```

const hook = () => {
  console.log('effect')
  axios
    .get('http://localhost:3001/notes')
    .then(response => {
      console.log('promise fulfilled')
      setNotes(response.data)
    })
}
useEffect(hook, []);
`**** useEffect(hook, []) ****`

```

- Now we can see more clearly that the function `useEffect` takes two parameters. The first is a function, the effect itself.
 - By default, effects run after every completed render, but you can choose to fire it only when certain values have changed.
- So by default, the effect is always run after the component has been rendered. In our case, however, we only want to execute the effect along with the first render.
- The second parameter of `useEffect` is used to specify how often the effect is run. If the second parameter is an empty array `[]`, then the effect is only run along with the first render of the component.
- Note that we could have also written the code for the effect function this way:

```

useEffect(() => {
  console.log('effect')

  const eventHandler = response => {
    console.log('promise fulfilled')
    setNotes(response.data)
  }

  const promise = axios.get('http://localhost:3001/notes')
  promise.then(eventHandler)
}, [])

```

- That is very clunky. It takes 3 steps and creates useless variables.

- Always use this shorter version unless major requirement

```
useEffect(() => {  
  console.log('effect')  
  axios  
    .get('http://localhost:3001/notes').then(response => {  
      console.log('promise fulfilled')  
      setNotes(response.data)  
    })  
}, [])
```

All About hook `useEffect(function,[])`;

- **Parameter 1: The Function (The "What")**

- The first parameter is just the code you want React to run.

Example: "Go fetch the data from the backend using Axios."

- **Parameter 2: The Array (The "When")**

- This is called the **Dependency Array**, and it is the most important part of `useEffect`. It tells React exactly **when it is allowed to run the function**. You have three choices:

- **Choice A: An Empty Array**

The **Empty Array []**, you pass an empty array, you are telling React: "Run this function exactly **ONE TIME** when the **website first loads**. Never run it again. This is perfect for fetching your initial database. You only need to download the notes once when the user opens the page!

- **Choice B: An Array with a variable inside [searchQuery]**

If you put a state variable inside the array, you are telling React: "Run this function when the page loads, **AND run it again every single time the searchQuery state changes**." (You will use this later in the course).

- **Choice C: No Array at all (DANGER 🚫)**

If you forget to type the array completely, you are telling React: "**Run this function every single time the component re-renders**." Because **fetching data updates the state, and updating the state causes a re-render**, this will cause a catastrophic **Infinite Loop** that will crash your browser.

The development runtime environment

1. The JavaScript code making up our React application is run in the browser.
2. The browser gets the JavaScript from the **React dev server**, which is the application that runs after running the command **npm run dev**
3. The dev-server transforms the JavaScript into a format understood by the browser.

4. Among other things, it stitches together JavaScript from different files into one file.
- The React application running in the browser fetches the JSON formatted data from json-server running on port 3001 on the machine. The server we query the data from - json-server - gets its data from the file db.json.

Final Verdict of this chapter (Course + Exercises)

- npm installs it permanently in your project, npx borrows it temporarily or executes it instantly.
- First always check for dependencies like axios or json-server.
- **npm install axios**
- **npm install json-server --save-dev** (directly get saved in dev dependencies) / for temporary **npx json-server --port 3001 db.json**
- **"server": "json-server -p 3001 db.json"** <= add this line into script section of the package.json
- **npm run server**
- react has habit or nature of re-rendering, if got indication (flare gun) from **useState** so it starts re-rendering all thing so this axios.get() method will get stuck into permanent loop so use empty array [] as a one time subscription for hook Effect.
- If we put **axios.get()** **fetch request** inside main.jsx (and put the ReactDOM.render() code inside the .then() block), we force React to sit there in the dark with its hands tied behind its back, **waiting for the API to finish its 5-second download**. The user sees nothing but a white screen.
- By putting the fetch inside with **useEffect**, React is allowed to draw rest of the components. Then, 5 seconds later, the data pops into the list.